ITA14 – ETHICAL HACKING

ASSIGNMENT 3 – CO3

Scan-Data Architecture, Validation, Cleaning, Transformation and Automation

Field Information
Mode Pair Work 2 Students per Group
Duration 1 Week
Total Marks 50
Student A G. Anjani Kumar (192373022)
Student B A. Guna Sekar (192525302)
Department / Class: slot B, CSE(DS) & AI
Faculty Name: Parthiban S

Academic submission – use only on authorized university lab systems.

General Instructions

Both students are responsible for understanding the complete assignment. The contribution labels indicate primary responsibilities, not exclusive work. Both students must participate in every question and be prepared to explain the complete workflow.

The examples use the illustrative authorized lab network 192.168.56.0/24. Replace sample observations, record counts and benchmark timings with actual lab results before submission.

Run scanning and enumeration commands only against systems and networks explicitly authorized by the university. Do not claim that a scan or enumeration was completed unless it was actually performed.

Q1 – Investigate Requirements and Create the Scan-Data Architecture (6 Marks)

Question

Translate the authorized-assessment scenario into service-scanning and enumeration requirements. Identify at least four scan datasets, design their schemas and relationships, create CSV headers, and formulate testable enquiry questions with acceptance criteria.

1.1 Requirements Analysis

The objective is to design a structured architecture that collects, organizes, validates and connects information obtained during an authorized security assessment.

- Discover active hosts using Nmap host discovery.
- Identify open, closed and filtered TCP/UDP ports.
- Detect running services and their versions.
- Identify operating systems and device types.
- Collect authorized enumeration information.
- Store scan results in separate CSV datasets.
- Connect related datasets using stable keys.
- Maintain scan timestamps, tool/source information and scan identifiers.

1.2 Scan Datasets and Schemas

Five datasets are designed. VARCHAR represents text, INTEGER represents whole numbers and DATETIME represents a date and time.

Dataset / Column	Data Type	Description
Hosts.ip	VARCHAR	IP address; host identifier
Hosts.hostname	VARCHAR	Device name
Hosts.status	VARCHAR	Up or Down

Hosts.discovery_method	VARCHAR	Nmap or fping
Hosts.first_seen	DATETIME	Discovery time
Hosts.scan_id	VARCHAR	Scan identification

Hosts.csv header:

```
ip,hostname,status,discovery_method,first_seen,scan_id
```

Dataset / Column	Data Type	Description
Ports.ip	VARCHAR	Host IP address
Ports.port	INTEGER	Port number, 1–65535
Ports.protocol	VARCHAR	TCP or UDP
Ports.state	VARCHAR	Open, Closed or Filtered
Ports.reason	VARCHAR	Scan response reason
Ports.scan_id	VARCHAR	Scan identification

Ports.csv header:

```
ip,port,protocol,state,reason,scan_id
```

Dataset / Column	Data Type	Description
Services.ip	VARCHAR	Host IP address
Services.port	INTEGER	Port number
Services.protocol	VARCHAR	TCP or UDP
Services.service	VARCHAR	Service name
Services.product	VARCHAR	Product name
Services.version	VARCHAR	Version information
Services.extra_info	VARCHAR	Additional information
Services.confidence	VARCHAR	Identification confidence
Services.scan_id	VARCHAR	Scan identification

Services.csv header:

```
ip,port,protocol,service,product,version,extra_info,confidence,scan_id
```

Dataset / Column	Data Type	Description
OS.ip	VARCHAR	Host IP address
OS.os_family	VARCHAR	Windows, Linux, Android etc.

OS.os_name	VARCHAR	Operating-system name
OS.os_version	VARCHAR	Version
OS.device_type	VARCHAR	Server, mobile, computer etc.
OS.accuracy	INTEGER	Detection accuracy, when available
OS.source	VARCHAR	Detection tool
OS.scan_id	VARCHAR	Scan identification

OS_Versions.csv header:

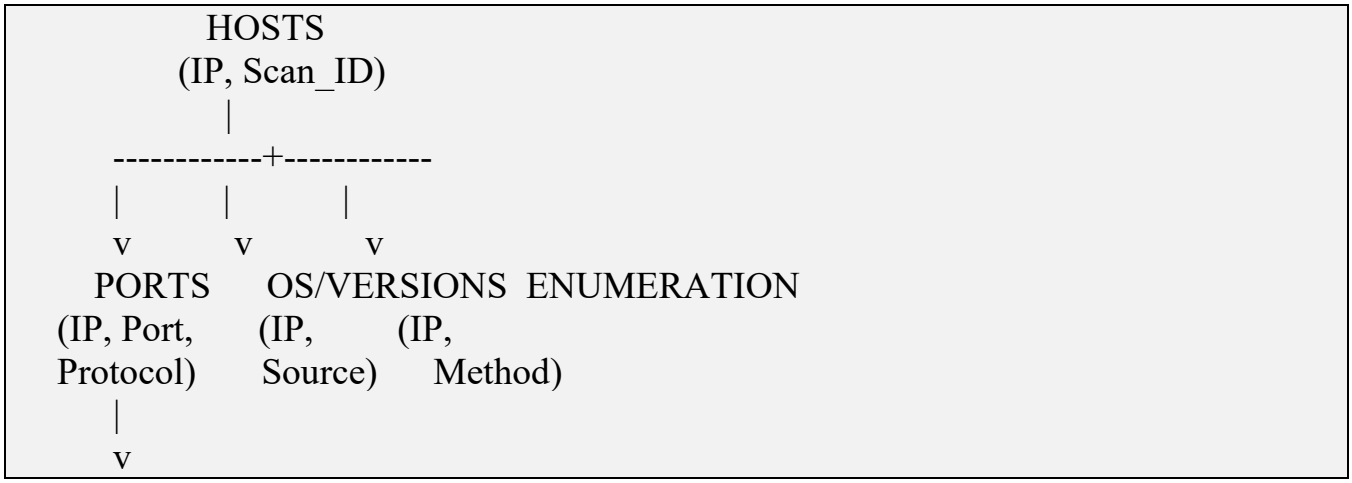
```
ip,os_family,os_name,os_version,device_type,accuracy,source,scan_id
```

Dataset / Column	Data Type	Description
Enumeration.ip	VARCHAR	Host IP address
Enumeration.port	INTEGER	Related port, if applicable
Enumeration.protocol	VARCHAR	TCP or UDP
Enumeration.method	VARCHAR	Enumeration tool
Enumeration.finding_type	VARCHAR	Type of finding
Enumeration.finding_value	VARCHAR	Collected information
Enumeration.status	VARCHAR	Observed, Not Tested or Review
Enumeration.scan_id	VARCHAR	Scan identification

Enumeration.csv header:

```
ip,port,protocol,method,finding_type,finding_value,status,scan_id
```

1.3 Relationship Diagram



```
    SERVICES
  (IP, Port,
   Protocol)
```

One host can have multiple port observations, OS observations and enumeration findings. Services are connected to ports using IP address, port number, protocol and scan ID. The scan ID distinguishes observations collected during different assessment sessions.

1.4 Requirements-to-Column Mapping

Requirement	Dataset Columns
Identify discovered systems	ip, hostname, status
Identify exposed ports	ip, port, protocol, state
Identify running services	service, product, version
Identify operating systems	os_family, os_name, os_version
Store enumeration results	method, finding_type, finding_value
Track assessment sessions	scan_id, first_seen

1.5 Testable Enquiry Questions

1. How many unique hosts were discovered?

```
awk -F, 'NR>1 {print $1}' hosts.csv | sort -u | wc -l
```

Acceptance: The result equals the number of unique valid IP addresses.

2. Which TCP ports are open?

```
awk -F, 'NR>1 && $3=="tcp" && $4=="open" {
   print $1 "," $2
}' ports.csv
```

Acceptance: Every returned port has a valid host IP and port number.

3. Which hosts are running HTTP services?

```
awk -F, 'NR>1 && tolower($4) ~ /http/ {
   print $1 "," $2
}' services.csv
```

Acceptance: Each service observation matches a corresponding host and endpoint in the Ports dataset.

4. Which hosts have unknown operating-system information?

```
awk -F, 'NR>1 && ($2=="" || tolower($2)=="unknown") {
    print $1
}' os_versions.csv
```

Acceptance: Unknown OS information is reported, not silently deleted.

1.6 Acceptance Criteria

Criterion	Explanation
Correctness	IP addresses, ports and service details must be valid.
Consistency	Service names and protocols follow standard naming.
Traceability	Each record contains scan ID and source information.
Reproducibility	Repeated processing of the same input produces the same output.

Student A: Host and Port schema design, requirements analysis and column mapping.

Student B: Service relationships, OS/Enumeration design and enquiry questions.

Q2 – Create and Validate the Source Scan Datasets (7 Marks)

Question

Construct realistic source scan datasets using Kali tools. Introduce controlled issues such as missing values, duplicate records, inconsistent service names and out-of-scope addresses. Develop validation and standardization checks before merging.

2.1 Host Discovery Using Nmap

Nmap host discovery identifies active devices in the approved assessment subnet.

```
sudo nmap -sn 192.168.56.0/24 -oA raw/discovery
fping -a -g 192.168.56.0/24 2>/dev/null
```

-sn performs host discovery without a port scan. -oA saves Nmap output in multiple formats. fping -a prints responding addresses.

2.2 Port and Service Scanning

```
# TCP connect scan
sudo nmap -sT -p 22,80,139,443,445 192.168.56.0/24
```

```
# TCP SYN scan
sudo nmap -sS -p 22,80,139,443,445 192.168.56.0/24

# UDP scan
sudo nmap -sU -p 53,123,161 192.168.56.0/24

# Service/version detection
sudo nmap -sV -p 22,80,139,443,445 192.168.56.0/24

# OS detection
sudo nmap -O 192.168.56.0/24
```

Use only the scan types and targets approved for the lab. Some scans require elevated privileges.

2.3 Enumeration Commands

Platform / Device	Tool	Purpose
Windows	enum4linux	SMB information
Windows	nbtscan	NetBIOS information
Android	adb	Android device properties
Java	rmi-dumpregistry	Java RMI registry information
NetWare	ncp-serverinfo	NetWare server information
Symbian-type Bluetooth device	sdptool	Bluetooth service discovery

```
enum4linux -a 192.168.56.10
nbtscan 192.168.56.0/24

adb devices
adb -s <authorized-device-id> shell getprop

rmi-dumpregistry <authorized-lab-host>:<approved-port>
ncp-serverinfo <authorized-lab-host>
sdptool browse <authorized-bluetooth-address>
```

These commands are conditional on the relevant lab devices, tools and services being available. Record unavailable platforms as Not Tested; do not create false findings.

2.4 Sample Source Datasets

IP	Hostname	Status	Discovery
192.168.56.10	WIN-LAB	Up	Nmap
192.168.56.20	ANDROID-EMU	Up	Nmap
192.168.56.30	JAVA-LAB	Up	Nmap

IP	Port	Protocol	State
192.168.56.10	80	TCP	Open
192.168.56.10	445	TCP	Open
192.168.56.30	1099	TCP	Open

IP	Port	Service	Product
192.168.56.10	80	http	Apache
192.168.56.10	445	microsoft-ds	SMB
192.168.56.30	1099	java-rmi	Java RMI

2.5 Controlled Defect Injection

Introduce defects only into working copies. Keep the original scan outputs unchanged.

```
cp clean/ports.csv work/ports_defective.csv

# Add a duplicate record for testing
tail -n 1 clean/ports.csv >> work/ports_defective.csv

# Add a documentation-only out-of-scope test address
echo '203.0.113.8,80,tcp,open,test,S001' >> work/ports_defective.csv

# Example service-name variation
sed 's/,http,/,HTTP,/' clean/services.csv \
  > work/services_inconsistent.csv
```

203.0.113.0/24 is reserved for documentation. The example line is synthetic test data, not a target to scan.

2.6 Validation Checks

Check the header:

```
head -n 1 clean/ports.csv
```

Check invalid port values:

```
awk -F, 'NR>1 {
   if ($2 !~ /^[0-9]+$/ || $2<1 || $2>65535)
      print "Invalid port:", NR, $2
}' clean/ports.csv
```

Check duplicate endpoint records:

```
tail -n +2 clean/ports.csv |
cut -d, -f1-3 | sort | uniq -d
```

Check approved subnet membership (install grepcidr if needed):

```
tail -n +2 clean/hosts.csv |
cut -d, -f1 |
grepcidr '192.168.56.0/24'
```

The command prints matching addresses. Use an inverse filter or compare the complete address list to identify records outside the approved subnet.

2.7 Service-Name Standardization

```
# service-map.csv
alias,standard
www,http
HTTP,http
http-alt,http
https,https
```

```
awk -F, 'BEGIN{OFS=","}
NR==FNR {
   if(FNR>1) map[$1]=$2
   next
}
FNR==1 {print; next}
{
   if($4 in map) $4=map[$4]
   print
}' work/service-map.csv clean/services.csv \
> work/services_standardized.csv
```

2.8 Which Validation Checks Should Be Applied First?

1. File and header validation: confirm required columns exist.
2. Scope validation: confirm every target belongs to the approved subnet.
3. Data-type validation: check IP format and port range.
4. Duplicate validation: detect repeated endpoint keys.
5. Standardization: normalize service names and protocol values.
6. Relationship validation: confirm service and port records refer to known hosts.

This order identifies structural and authorization problems before data is cleaned or merged.

Student A: Host/Port construction and defect injection. Student B: Services, OS/Enumeration construction and validation.

Q3 – Build a Scan-Data Editing and Cleaning Module (7 Marks)

Question

Develop reusable shell operations using Bash functions, sed, awk and sort -u to handle missing values, inconsistent service names, duplicate host-port records and malformed addresses. Show before-and-after data and test valid, invalid and boundary cases.

3.1 Purpose and Cleaning Rules

Scan-data cleaning identifies and corrects errors before analysis. Missing critical identifiers should not be guessed; ambiguous records must be preserved for manual review.

Data Problem	Cleaning Operation
Missing hostname	Replace optional missing hostname with unknown or retain blank with a review flag.
Duplicate endpoint	Identify duplicate IP, port, protocol and scan-session keys; log removed duplicates.
Invalid IP address	Reject the record and preserve it in a rejected file.
Invalid port number	Reject the record and preserve it in a rejected file.
Inconsistent service name	Standardize through an approved lookup file.
Unknown service name	Preserve it and mark for manual review.

3.2 Reusable Bash Validation Functions

```
#!/bin/bash

check_ipv4() {
   awk -v ip="$1" 'BEGIN {
      n=split(ip,a,".")
      if(n!=4) exit 1
      for(i=1;i<=4;i++)
         if(a[i]!~/^[0-9]+$/ || a[i]<0 || a[i]>255)
            exit 1
   }'
}

check_port() {
   [[ "$1" =~ ^[0-9]+$ ]] &&
   (( $1 >= 1 && $1 <= 65535 ))
}

check_ipv4 192.168.56.10
echo "IP validation status: $?"

check_port 80
echo "Port validation status: $?"
```

A return status of 0 indicates valid input. A non-zero return status indicates invalid input.

3.3 Duplicate Detection

```
# Display duplicate complete rows after sorting
tail -n +2 clean/ports.csv | sort | uniq -d

# Detect duplicate endpoint keys: IP, port and protocol
tail -n +2 clean/ports.csv |
cut -d, -f1-3 | sort | uniq -d
```

Do not deduplicate only by IP address: one host can legitimately have many ports. Include protocol and scan/session identity in the key when comparing sessions.

3.4 Missing-Value Handling

```
awk -F, 'NR>1 && $4=="" {
   print "Missing service:", NR, $1, $2
}' clean/services.csv
```

```
awk -F, 'BEGIN{OFS=","}
NR==1 {print; next}
$2=="" {$2="unknown"}
{print}' clean/hosts.csv > work/hosts_clean.csv
```

An absent IP address or port is a critical missing value and should be rejected or manually reviewed rather than invented.

3.5 Service-Name Standardization

```
# service-map.csv
alias,standard
www,http
HTTP,http
http-alt,http
https,https
```

```
awk -F, 'BEGIN{OFS=","}
NR==FNR {
   if(FNR>1) map[$1]=$2
   next
}
FNR==1 {print; next}
{
   if($4 in map) $4=map[$4]
   print
}' work/service-map.csv clean/services.csv \
> work/services_clean.csv
```

3.6 Boundary Test Cases

Test Input	Expected Result	Reason
Port 0	Invalid	Below valid range
Port 1	Valid	Lowest valid port
Port 65535	Valid	Highest valid port
Port 65536	Invalid	Above valid range
192.168.56.255	Valid IPv4 format	All octets are within range
192.168.56.256	Invalid	Last octet exceeds 255

```
for p in 0 1 65535 65536
```

```
do
   if check_port "$p"; then
      echo "$p : VALID"
   else
      echo "$p : INVALID"
   fi
done

for ip in 192.168.56.255 192.168.56.256
do
   if check_ipv4 "$ip"; then
      echo "$ip : VALID"
   else
      echo "$ip : INVALID"
   fi
done
```

The address 192.168.56.255 is syntactically a valid IPv4 address, but in a /24 network it is the broadcast address and should not be treated as an ordinary host target.

3.7 Before-and-After Verification

```
wc -l work/ports_defective.csv
head work/ports_defective.csv

wc -l work/ports_clean.csv
head work/ports_clean.csv

diff -u work/ports_defective.csv work/ports_clean.csv
```

Every removed record must be represented in a duplicate or rejected log. Reconcile input records against accepted, rejected and duplicate records. Exclude headers when comparing data-record totals.

3.8 Why Is Automated Cleaning Alone Insufficient?

Automated cleaning cannot understand every security observation. A new service name may be valid even if it is not in the lookup file. Two records with the same IP and port may belong to different scan sessions. Nmap may identify an operating system with low confidence. A missing value may mean a device was unavailable rather than that the record is erroneous. Therefore, automated cleaning must be combined with manual review, raw-data preservation and validation logs.

Student A: Missing values, duplicate records and malformed IP correction. Student B: Service-name standardization and verification tests.

Q4 – Create a Scan-Result Reshaping Pipeline Using Melting and Casting (7 Marks)

Question

Create a wide-format scan dataset with hosts as rows and ports as columns. Convert it into long format using awk, create a cast operation and validate that the transformation preserves the original information.

4.1 Introduction

Melting converts wide-format data into long-format data. Casting converts long-format data into a summarized wide-format structure.

4.2 Wide-Format Dataset

IP Address	21	22	23	25	80	443	445
192.168.56.10	closed	open	closed	closed	open	open	open
192.168.56.20	closed	closed	closed	closed	closed	closed	closed
192.168.56.30	closed	open	closed	closed	closed	closed	closed

Identifier: IP address. Dimension: port number. Measured variable: port state.

```
ip,21,22,23,25,80,443,445
192.168.56.10,closed,open,closed,closed,open,open,open
192.168.56.20,closed,closed,closed,closed,closed,closed,closed
192.168.56.30,closed,open,closed,closed,closed,closed,closed
```

4.3 Wide-to-Long Transformation (Melting)

```
awk -F, 'BEGIN {OFS=","}
NR==1 {
   for(i=2;i<=NF;i++) port[i]=$i
   print "ip,port,state"
   next
}
{
   for(i=2;i<=NF;i++)
      print $1,port[i],$i
}' wide.csv > long.csv
```

Sample long-format output:

```
ip,port,state
192.168.56.10,21,closed
192.168.56.10,22,open
192.168.56.10,23,closed
192.168.56.10,25,closed
192.168.56.10,80,open
192.168.56.10,443,open
192.168.56.10,445,open
192.168.56.20,21,closed
192.168.56.20,22,closed
192.168.56.30,22,open
```

The complete output contains 21 data rows because there are 3 hosts and 7 port columns.

4.4 Long-to-Wide Summary (Casting)

```
awk -F, 'NR>1 && $3=="open" {
    count[$1]++
}
END {
    print "IP,Open_Port_Count"
    for(ip in count)
        print ip "," count[ip]
}' long.csv | sort > open_ports.csv
```

This simple example omits hosts with zero open ports. For a complete management report, initialize every host to zero from the Hosts dataset before incrementing its open-port count.

4.5 Validation of Reshaping

```
wc -l wide.csv
wc -l long.csv

sha256sum long.csv
cp long.csv long_copy.csv
sha256sum long_copy.csv
```

Expected long-format data rows = number of hosts × number of port columns = 3 × 7 = 21. The CSV header adds one physical line. A robust round-trip test should reconstruct the wide table and compare normalized values or checksums of canonicalized files; hashing an unchanged copy only verifies the copy.

4.6 Why More Detail Does Not Always Produce a Clearer Summary

A full TCP port scan can examine up to 65,535 ports per host. For 100 hosts, this creates up to 6,553,500 host-port observations. Large datasets increase scan duration, storage, processing time and analysis complexity. Management summaries should highlight exposed services and verified version risks while retaining complete raw data for technical investigation.

```
time nmap -p 21,22,23,25,80,443,445 192.168.56.0/24
wc -l long.csv
```

Record actual scan time and row counts. Student A: Wide-to-long transformation and dimension identification. Student B: Casting, summary generation and validation.

Q5 – Design a Multi-Stage Scan-Data Merging Strategy (6 Marks)

Question

Combine the Hosts, Ports, Services, OS/Versions and Enumeration datasets into a master scan table. Identify suitable keys, compare join types and detect unmatched records and unexpected row multiplication.

5.1 Introduction

Data merging combines related scan datasets into a master table so analysts can understand discovered hosts, exposed ports, services, operating systems and enumeration findings.

5.2 Dataset Relationship Keys

Relationship	Recommended Key
Hosts to Ports	IP address + Scan ID
Ports to Services	IP + Port + Protocol + Scan ID
Hosts to OS/Versions	IP + Source + Scan ID
Hosts to Enumeration	IP + Method + Scan ID

TCP port 80 and UDP port 80 are different endpoints. Include protocol in endpoint keys. Include scan/session identity to distinguish repeated observations.

5.3 Sorting Before Joining

```
tail -n +2 hosts.csv |
sort -t, -k1,1 > hosts_sorted.csv
```

```
tail -n +2 ports.csv |
sort -t, -k1,1 > ports_sorted.csv
```

GNU join requires inputs sorted by the selected join field. The following examples use simplified one-column joins on IP; production joins should use validated composite keys where needed.

5.4 Inner, Left and Full Outer Joins

Inner join – returns matching records only:

```
join -t, -1 1 -2 1 hosts_sorted.csv ports_sorted.csv
```

Left join – preserves every record from the first dataset:

```
join -t, -1 1 -2 1 -a1 hosts_sorted.csv ports_sorted.csv
```

Full outer join – preserves matching and unmatched records from both datasets:

```
join -t, -1 1 -2 1 -a1 -a2 hosts_sorted.csv ports_sorted.csv
```

5.5 Detect Unmatched Records

Hosts without matching port records:

```
join -t, -1 1 -2 1 -v1 hosts_sorted.csv ports_sorted.csv
```

Port records without matching hosts:

```
join -t, -1 1 -2 1 -v2 hosts_sorted.csv ports_sorted.csv
```

5.6 Detect Duplicate Records and Validate Counts

```
sort ports.csv | uniq -d

tail -n +2 ports.csv |
cut -d, -f1-3 | sort | uniq -d

wc -l hosts_sorted.csv
wc -l ports_sorted.csv
wc -l host_ports_joined.csv
```

A join on IP alone can multiply rows. For example, five port rows and three enumeration findings for one host may produce 15 combinations. Use composite keys or summarize child datasets before combining them into a host-level report.

5.7 Compare Join Types

Join Type	Output
Inner join	Only matching records
Left join	All left-side records and matching right-side records
Full outer join	All records from both sides
Unmatched join	Records without a matching key

5.8 Cross-Understanding Answers

Why can different join types produce different record counts? Different joins retain different groups of records. Inner joins remove unmatched rows, left joins preserve all left-side rows, and full outer joins preserve unmatched rows from both sides.

How can unmatched records be detected? Use join -v1, join -v2 and comm to identify keys absent from the other dataset.

What evidence demonstrates that a join is correct? Correct keys, sorted inputs, duplicate checks, recorded unmatched records, reconciled row counts and no unexplained row multiplication.

Student A: Host-Service and Ports-to-Hosts integration. Student B: OS/Versions and Enumeration integration and exception analysis.

Q6 – Create an Automated Scan-File Reading and Writing Pipeline (7 Marks)

Question

Develop a pipeline that reads scan-output files, validates required columns, imports results into CSV tables, exports cleaned and merged datasets, applies versioning and verifies the written output.

6.1 Introduction

An automated scan-file pipeline reads source outputs, validates their structure, imports scan data, cleans and processes records, exports final datasets and verifies the exported files.

6.2 Reading Nmap Scan Results

```
sudo nmap -sV 192.168.56.0/24 -oX scan.xml
sudo nmap -sV 192.168.56.0/24 -oG scan.gnmap

grep '^Host:' scan.gnmap
```

Nmap XML is structured and suitable for parsing with an XML-aware parser. Grepable output can be processed for simple lab demonstrations. Preserve host, port, protocol, state, service and version fields during conversion.

6.3 Required-Column Validation

```
#!/bin/bash
file="$1"

if [[ ! -r "$file" ]]; then
    echo "ERROR: File not found" >&2
    exit 2
fi

expected="ip,port,protocol,state,reason,scan_id"
actual=$(head -n 1 "$file")

if [[ "$actual" != "$expected" ]]; then
    echo "ERROR: Header mismatch" >&2
    exit 3
fi

echo "Header validation successful"
```

6.4 Data-Type Validation

```
awk -F, 'NR>1 {
    if ($2 !~ /^[0-9]+$/ || $2<1 || $2>65535)
        print "Invalid port at line:", NR
}' ports.csv
```

6.5 Output Naming and Versioning

Use a naming convention such as subnet_scantype_YYYYMMDD-HHMMSS_vN.csv. Example: 192.168.56.0-24_tcp_20261103-100000_v1.csv. This identifies subnet, scan type, timestamp and version.

6.6 Prevent Accidental Overwriting

```
set -o noclobber
output="scan_results.csv"

if ! ( : > "$output" ); then
   echo "File already exists. Choose another filename."
   exit 1
fi
```

This example deliberately fails if the output file already exists. In a production workflow, generate a unique timestamp/versioned name and check that it does not exist before writing.

6.7 Generate SHA-256 Checksum and Read-Back Validation

```
sha256sum scan_results.csv > scan_results.csv.sha256

cp scan_results.csv scan_results_copy.csv
cmp scan_results.csv scan_results_copy.csv
sha256sum scan_results.csv scan_results_copy.csv
wc -l scan_results.csv scan_results_copy.csv
diff -u scan_results.csv scan_results_copy.csv
```

If the files are identical, cmp and diff report no differences and their SHA-256 values match. Use sha256sum -c scan_results.csv.sha256 to verify a stored checksum.

6.8 Importance of Automated File Handling

- Prevents accidental data loss.
- Maintains multiple scan versions.
- Improves data consistency.
- Identifies invalid file formats.
- Supports repeatable analysis.
- Makes scan results easier to audit.

Student A: Scan-file import and schema validation. Student B: Result export, naming, versioning and read-back validation.

Q7 – Develop a Reusable Scanning and Enumeration Function Library (10 Marks)

Question

Develop a reusable Bash function library named scanlib.sh supporting validation, cleaning, reshaping, merging, file handling, reporting, benchmarking and multi-session comparison.

7.1 Introduction

A reusable Bash function library contains commonly used operations that can be loaded and executed whenever required. The proposed library is named scanlib.sh and supports validation, cleaning, reshaping, merging, file export, reporting, benchmarking and session comparison.

Stage 1 – Independent Function Design Proposal

Student A – Validation and Cleaning Function Design

Function	Purpose
valid_ipv4()	Checks IPv4 addresses
valid_port()	Checks port numbers
validate_ports_csv()	Validates port CSV records
dedupe_ports()	Identifies duplicate endpoints

```
valid_port() {
    [[ "$1" =~ ^[0-9]+$ ]] &&
    (( $1 >= 1 && $1 <= 65535 ))
}
```

Student B – Scan Transformation and File-Handling Function Design

Function	Purpose
scan_discovery()	Performs authorized host discovery
melt_wide()	Converts wide data to long data
merge_tables()	Combines related datasets
export_versioned()	Exports files with version information

```
melt_wide() {
    awk -F, 'BEGIN {OFS=","}
```

```
    NR==1 {
        for(i=2;i<=NF;i++) port[i]=$i
        print "ip,port,state"
        next
    }
    {
        for(i=2;i<=NF;i++)
            print $1,port[i],$i
    }' "$1" > "$2"
}
```

Both students must retain their initial proposals in the submission. An initial design may be revised during integration; the purpose is to show independent algorithmic thinking.

Stage 2 – Integrated Pipeline Implementation and Validation

Create the library:

```
nano scanlib.sh
```

Enter the following implementation:

```
#!/usr/bin/env bash
# Reusable scan-data function library

set -o pipefail

die() {
    echo "ERROR: $*" >&2
    return 1
}

require_file() {
    [[ -r "$1" ]] || {
        die "File not found: $1"
        return 2
    }
}

valid_ipv4() {
    awk -v ip="$1" 'BEGIN {
        n=split(ip,a,".")
        if(n!=4) exit 1
        for(i=1;i<=4;i++)
```

```
            if(a[i]!~/^[0-9]+$/ || a[i]<0 || a[i]>255)
                exit 1
    }'
}

valid_port() {
    [[ "$1" =~ ^[0-9]+$ ]] &&
    (( $1 >= 1 && $1 <= 65535 ))
}

validate_ports_csv() {
    input="$1"
    valid="$2"
    rejected="$3"

    require_file "$input" || return $?

    head -n 1 "$input" > "$valid"
    echo "reason,record" > "$rejected"

    tail -n +2 "$input" |
    while IFS=, read -r ip port proto state reason scanid
    do
        row="$ip,$port,$proto,$state,$reason,$scanid"

        if ! valid_ipv4 "$ip"; then
            echo "INVALID_IP,$row" >> "$rejected"
        elif ! valid_port "$port"; then
            echo "INVALID_PORT,$row" >> "$rejected"
        elif [[ "$proto" != tcp && "$proto" != udp ]]; then
            echo "INVALID_PROTOCOL,$row" >> "$rejected"
        else
            echo "$row" >> "$valid"
        fi
    done
}

melt_wide() {
    require_file "$1" || return $?

    awk -F, 'BEGIN {OFS=","}
    NR==1 {
```

```
        for(i=2;i<=NF;i++) p[i]=$i
        print "ip,port,state"
        next
    }
    {
        for(i=2;i<=NF;i++)
            print $1,p[i],$i
    }' "$1" > "$2"
}

host_open_summary() {
    require_file "$1" || return $?

    {
        echo "ip,open_port_count"
        awk -F, 'NR>1 && $3=="open" {
            count[$1]++
        }
        END {
            for(ip in count)
                print ip "," count[ip]
        }' "$1" | sort
    } > "$2"
}

export_versioned() {
    require_file "$1" || return $?
    mkdir -p "$2" || return 5

    timestamp=$(date -u +%Y%m%d-%H%M%S)
    target="$2/${3:-scan}_${timestamp}_v1.csv"

    [[ ! -e "$target" ]] || {
        die "Refusing to overwrite existing file"
        return 6
    }

    cp -- "$1" "$target" || return 7
    sha256sum "$target" > "$target.sha256"
    echo "$target"
}
```

```
scan_discovery() {
   subnet="${1:?Approved subnet required}"
   output="${2:-discovery.xml}"

   command -v nmap >/dev/null || {
      die "Nmap is not installed"
      return 8
   }

   [[ "$subnet" == "192.168.56.0/24" ]] || {
      die "Subnet is not in configured example scope"
      return 9
   }

   sudo nmap -sn "$subnet" -oX "$output"
}
```

This is a teaching example for a simple, unquoted CSV format. Production CSV files with quoted commas or multiline fields require a CSV-aware parser. The discovery function is deliberately restricted to the illustrative lab subnet. Adapt scope only after written authorization.

7.3 Load and Test the Function Library

```
source ./scanlib.sh

valid_ipv4 192.168.56.10
echo $?

valid_port 80
echo $?

valid_port 65536
echo $?
```

Expected status codes: valid IP and port return 0; invalid port 65536 returns a non-zero status.

7.4 Execute the Integrated Pipeline

```
source ./scanlib.sh

mkdir -p clean reports archive
```

```
validate_ports_csv \
  clean/ports.csv \
  clean/ports_valid.csv \
  clean/ports_rejected.csv

melt_wide clean/wide.csv clean/long.csv

host_open_summary clean/long.csv reports/host_summary.csv

export_versioned reports/host_summary.csv archive host_summary
```

7.5 Record Input, Rejected and Cleaned Records

```
wc -l clean/ports.csv
wc -l clean/ports_valid.csv
wc -l clean/ports_rejected.csv
```

Measurement	Record Count
Input records	Enter actual lab result
Valid records	Enter actual lab result
Rejected records	Enter actual lab result
Duplicate records	Enter actual lab result
Final cleaned records	Enter actual lab result

Exclude header lines when calculating data-record totals. Account separately for duplicates if they are also included in rejected records, so the reconciliation does not count them twice.

Stage 3 – Create and Validate a Reporting Scan Summary

Student A – Host and Service Reporting

```
# Open ports per host; for Ports.csv layout
awk -F, 'NR>1 && $4=="open" {
   count[$1]++
}
END {
   for(ip in count)
      print ip "," count[ip]
}' clean/ports.csv | sort
```

```
# Service frequency; for Services.csv layout
awk -F, 'NR>1 {
   count[$4]++
}
END {
   for(service in count)
      print service "," count[service]
}' clean/services.csv | sort
```

Illustrative service report: HTTP – 5 observations; HTTPS – 3; SMB – 2; SSH – 4. Replace these figures with actual output. Reconcile report totals with the number of valid service records included.

Student B – Vulnerability-Exposure Reporting

```
sudo nmap -sV -p 80,443,445 <approved-lab-host>

# Only when the vulnerability scripts are specifically authorized
sudo nmap -sV --script vuln -p 80,443,445 <approved-lab-host>

searchsploit <product-name> <version>
```

Example version-review report:

```
awk -F, 'NR>1 &&
($6=="" || tolower($6)~/unknown|old|legacy/) {
   print $1 "," $2 "," $4 "," $5 "," $6
}' clean/services.csv > reports/version_review.csv
```

Field	Description
Host IP	Affected system
Port	Exposed endpoint
Service	Running service
Product	Software product
Version	Detected version
Risk	Verified security concern
Recommendation	Suggested corrective action

The report helps the university IT department identify exposed services and prioritize review. An old version string or a Searchsploit result does not independently prove a vulnerability; verify applicable advisories, patch status and configuration.

Stage 4 – Stress-Test, Refine and Optimize the Scan Pipeline

Student A – Dataset Scaling and Baseline Benchmarking

```
# Create 50 synthetic records
echo "ip,port,protocol,state,reason,scan_id" > small.csv
for i in $(seq 1 50); do
  echo "192.168.56.10,$i,tcp,open,test,SYN01" >> small.csv
done

# Create 5,000 synthetic records
echo "ip,port,protocol,state,reason,scan_id" > large.csv
for i in $(seq 1 5000); do
  echo "192.168.56.10,$i,tcp,open,test,SYN02" >> large.csv
done

/usr/bin/time -v wc -l small.csv
/usr/bin/time -v wc -l large.csv
```

Dataset	Number of Records	Elapsed Time	Memory
Small	50	Actual measurement	Actual measurement
Large	5,000	Actual measurement	Actual measurement

These are synthetic processing datasets, not actual port scan results. Port numbers above 65535 in a synthetic test should not be treated as valid scan records; use a separate synthetic row identifier if testing 5,000 records.

Student B – Bottleneck Analysis and Optimization

```
# One-pass service aggregation using awk
awk -F, 'NR>1 {count[$4]++}
END {for(s in count) print s,count[s]}' large_services.csv

# Measure execution time
time awk -F, 'NR>1 {count[$4]++}
END {for(s in count) print s,count[s]}' large_services.csv
```

A repeated grep inside a per-line loop starts many external processes. A single awk pass can reduce command overhead. Compare before-and-after outputs with diff or checksums; speed improvement is useful only when correctness is preserved.

Stage 5 – Redesign the Pipeline Under a Changed Assessment Requirement

Student A – Schema Evolution and Architecture Redesign

When UDP scanning is introduced, the endpoint key must include protocol.

```
IP + Port + Protocol + Scan_ID
```

Revised datasets may include:

Dataset	Additional Fields
Hosts	asset_id, device_family, scope_id
Ports	protocol, scan_id, observed_at
Services	product, version, confidence
OS/Versions	source, accuracy, device_type
Enumeration	method, finding_type, finding_value
Scan Sessions	scan_id, tool_version, start_time, end_time

Newly discovered Windows, Android, Java, NetWare and Symbian-type devices can be stored without changing the core host structure. Preserve the source and method used to identify each device.

Student B – Multi-Session Workflow Implementation

```
ndiff raw/session1.xml raw/session2.xml

# Retained hosts
comm -12 \
  <(sort -u work/session1_hosts.txt) \
  <(sort -u work/session2_hosts.txt)

# Newly discovered hosts
comm -13 \
  <(sort -u work/session1_hosts.txt) \
  <(sort -u work/session2_hosts.txt)

# Previously observed but not seen in session 2
comm -23 \
  <(sort -u work/session1_hosts.txt) \
  <(sort -u work/session2_hosts.txt)
```

Category	Meaning

Retained hosts	Present in both sessions
Newly discovered hosts	Present only in the second session
No longer observed	Present in first but not second session
Updated hosts	Host information or services changed

A host no longer observed is not necessarily removed from the network. It may be offline or filtered during the second scan.

Stage 6 – Joint Evaluation

7.6 Why Is a Structured Sequence Required?

```
Scan Data Import
      |
      v
Validation
      |
      v
Cleaning
      |
      v
Reshaping
      |
      v
Merging
      |
      v
Reporting
      |
      v
Export
```

Stage	Purpose
Import	Reads scan results from source files.
Validation	Checks columns, IP addresses and port numbers.
Cleaning	Handles missing, duplicate and inconsistent values.
Reshaping	Converts data into suitable formats.
Merging	Connects related scan datasets.

Reporting	Generates host, service and exposure summaries.
Export	Saves final results with versioning and checksums.

7.7 Benefits of Modular Functions

- Reusable functions reduce repeated code.
- Separate modules make debugging easier.
- Validation functions improve data quality.
- Error handling prevents unexpected failures.
- Versioning protects earlier scan results.
- Modular design makes future updates easier.

7.8 Additional Costs and Challenges

Challenge	Explanation
Processing time	Large datasets require more time.
Storage	Multiple scan sessions consume storage.
Validation	Checking every record requires processing.
Error handling	More conditions increase code complexity.
Repeated scanning	Network and device load may increase.
Integration	Different tools may produce different formats.
Manual review	Ambiguous findings require analyst attention.

7.9 When Can the Pipeline Become Unsuitable?

- The number of hosts and scan records becomes extremely large.
- Different tools produce incompatible formats.
- The dataset contains many missing or malformed records.
- Network devices change frequently.
- Scan sessions use inconsistent timestamps or identifiers.
- The schema does not support new protocols or device types.
- The system cannot maintain accurate source information.

7.10 Joint Conclusion

A reliable scan-data pipeline requires a structured sequence of importing, validating, cleaning, reshaping, merging and exporting operations. Each stage performs a specific task and helps maintain the accuracy of the final security assessment.

Reusable Bash functions improve maintainability, reduce repeated code and simplify testing. Systematic transformations help analysts connect host, port, service, operating-system and enumeration information.

Automated processing alone cannot guarantee accurate findings. Manual verification, raw-data preservation, proper authorization, error handling and reproducibility checks are also essential. A modular and maintainable pipeline helps security analysts perform efficient assessments while preserving the correctness and traceability of scan results.

CONTRIBUTION RECORD

The contribution record must reflect the actual work completed. Primary contribution does not mean exclusive responsibility.

Task	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1	Host and Port schema design	Service relationships and enquiry design	Architecture and acceptance criteria
Q2	Host/Port construction and defect injection	Services, OS and Enumeration datasets; validation	Validation strategy
Q3	Missing values, duplicates and IP correction	Service-name standardization and tests	Cleaning rules and verification
Q4	Wide-to-long transformation	Casting and summary validation	Reshaping design
Q5	Host-Service and Ports-to-Hosts integration	OS/Enumeration integration	Master table verification
Q6	File import and schema validation	Export, versioning and read-back validation	Pipeline testing
Q7	Validation, cleaning, host reporting, baseline	Transformation, merging, export, exposure reporting,	Integrated function library and joint evaluation

	benchmarking and schema redesign	optimization and multi-session workflow	

Student A Signature: G. Anjani Kumar
Student B Signature: A. Guna Sekar

Screenshots and Evidence to Attach

- Nmap host discovery output.
- TCP and UDP port scan results.
- Nmap service and version detection.
- Sample Hosts, Ports and Services CSV files.
- OS/Versions and Enumeration datasets.
- Defective CSV files showing missing values and duplicates.
- Validation results for IP addresses and port numbers.
- Before-and-after cleaning results.
- Wide-format and long-format datasets.
- Casting and per-host summary output.
- Join results and unmatched records.
- File export, versioning and SHA-256 verification.
- scanlib.sh function library.
- Benchmark results for 50 and 5,000 records.
- Multi-session comparison using ndiff and comm.
- Final host, service and exposure reports.

References

7. Nmap Reference Guide: https://nmap.org/book/man.html
8. Nmap Output Formats: https://nmap.org/book/output-formats.html
9. GNU Bash Reference Manual: https://www.gnu.org/software/bash/manual/
10. GNU Coreutils Manual: https://www.gnu.org/software/coreutils/manual/
11. Kali Linux Tools: https://www.kali.org/tools/